

Class Notes: Day 2 - Session 5 - Database Commands

Managing Large Databases Using MongoDB

Date: August 19, 2025 | Time: 01:31 PM WAT

Concept 1: Introduction to Database Commands

Definition:

Database commands in MongoDB are operations that allow developers to create, modify, and update databases, users, collections, and perform other administrative tasks. They are executed using helpers like `db.runCommand()` or `db.adminCommand()`.

Explanation:

Database commands are essential for managing MongoDB instances. They cover categories such as user management, authentication, write operations, and aggregation. Commands are run by passing a document or string to `db.runCommand()`, which provides a consistent interface across the shell and drivers. For administrative commands, `db.adminCommand()` can be used to target the admin database.

Example:

To run a simple command like getting the server status:

```
db.runCommand({ serverStatus: 1 })
```

This returns server metrics. From the book, commands are demonstrated using `db.runCommand()` for user creation and other tasks.

Class Work:

1. Connect to a MongoDB shell and run `db.runCommand({ ping: 1 })` to check if the server is alive.
2. Switch to the admin database and run an administrative command like

```
db.adminCommand({ listDatabases: 1 })
```

.
3. Discuss the output and its usefulness.

Concept 2: User Management Commands

Definition:

User management commands in MongoDB handle operations related to users, including creation, role assignment, information retrieval, and deletion.

Explanation:

These commands ensure secure access control. Key commands include `createUser` for adding new users, `grantRolesToUser` for assigning roles, `usersInfo` for viewing user details, and `dropUser` for removing users. Roles like `readWrite` or `read` define permissions.

Example:

Using the `sample_training` database from the book:

Create a user:

```
db.runCommand({ createUser: "User_1", pwd: "user1", roles: ["readWrite"] })
```

Grant additional role:

```
db.runCommand({ grantRolesToUser: "User_1", roles: [{ role: "read", db: "sample_supplies" }] })
```

View user info:

```
db.runCommand({ usersInfo: { user: "User_1", db: "sample_training" }, showPrivileges: true })
```

Drop user:

```
db.runCommand({ dropUser: "User_1" })
```

Class Work:

1. Create a new user "db_user" with password "pass123" and role "readWrite" in a test database.
2. Grant the "read" role on another database to this user.
3. Use `usersInfo` to view the user's details and privileges.
4. Drop the user and verify using `usersInfo`.

Concept 3: Authentication Commands

Definition:

Authentication commands verify user credentials against stored values to ensure secure access to the database.

Explanation:

The primary command is `db.auth()`, which authenticates a user and returns 1 on success or 0 on failure. It supports mechanisms like x.509 and prompts for passwords securely.

Example:

Authenticate User_1 from the book:

```
db.auth("User_1", "user1")
```

Output: 1 (success). If wrong password: 0.

Class Work:

1. Create a user if not exists, then authenticate using `db.auth()` with correct and incorrect passwords.
2. Use password prompt: `db.auth("User_1", passwordPrompt())`.
3. Discuss security implications of authentication in MongoDB.